EX06 - Factorial - Product Accumulator and Descending range

Beginner Step-by-Step Guide

This guide follows the real file `ex06_factorial.py`. It explains the problem first and then the code from the perspective of someone starting Python.

1. What we want to solve

Compute n! in ascending and descending order and compare it with math.factorial().

Problem variables / elements

Name	Meaning
n	non-negative integer
fact	running product

Equation or logic on paper

```
n! = n*(n-1)*...*1; 0! = 1
```

Manual check

For n=5: 1*2*3*4*5=120 and 5*4*3*2*1=120.

2. What we already know and what is new

Already seen:

- for
- range
- isinstance()
- accumulators

New in this exercise:

- from math import factorial
- product starts at 1
- *=
- range with step=-1

3. New Python concepts

1. Selective import

Syntax: `from math import factorial`

We import only factorial and can call factorial(n) directly.

2. Product accumulator

Syntax: `fact = 1`

A product starts at 1 because multiplying by 1 leaves the value unchanged.

3. *=

Syntax: `fact *= i`

Equivalent to fact = fact * i.

4. Descending range

Syntax: `range(n, 0, -1)`

The third argument is the step. -1 makes i decrease; stop=0 is excluded.

4. Code step by step

Step 1. Initialize products

```
fact = 1
fact_reverse = 1
```

Two accumulators demonstrate that both orders produce the same factorial.

Step 2. Ascending

```
for i in range(1, n + 1):
    fact *= i
```

i becomes 1,2,...,n and fact stores the partial product.

Step 3. Descending

```
for i in range(n, 0, -1):
    fact_reverse *= i
```

Now i starts at n and decreases to 1.

Step 4. Validate with library

```
analytical_result = factorial(n)
```

The math function provides an independent reference.

5. Execution flow

```
n
 -> validate
 -> product upward
 -> product downward
 -> math.factorial reference
```

6. Expected result / check

With n=5, fact=120, fact_reverse=120 and factorial(5)=120.

7. Common beginner mistakes

- Starting fact=0.
- Using range(n,0) without a negative step.
- Forgetting 0!=1.
- Confusing *= with **=.

8. If you remember only five things

- A sum starts at 0; a product at 1.
- range accepts start, stop and step.
- *= multiplies and reassigns.
- Order does not change the factorial.
- The library provides a reference.

9. Mini-practice

Change one input. Before running, write a comment predicting what should change and why. Then run the file and compare your prediction with the output.

Completion criterion

Move on when you can explain the problem, equation/logic, new Python concepts and execution flow without simply reading the code.